

Findings

This document answers the six questions from the assignment brief: the rules I implemented, the false positives and false negatives to expect, what I would improve with more time, the security and privacy concerns when scanning source code, and how I would prioritize findings in practice.

1. What rules did you implement?

I have implemented fifteen rules, plus an opt-in dependency check. I implemented the ones suggested in the brief and added several that come up often in real code.

Suggested:

- Hardcoded secrets (`api_key`, `password`, `token`, ...)
- Dangerous functions (`eval`, `exec`, `subprocess(..., shell=True)`, `os.system`)
- Insecure `http://` URLs
- Debug config (`debug=true`, `NODE_ENV=development`)
- Broad CORS (`origin: "*"`)
- Suspicious security comments (`TODO security`, `FIXME auth`, `temporary bypass`)

Added:

- Private key blocks (`-----BEGIN ... PRIVATE KEY-----`)
- High-entropy strings next to a secret keyword
- Insecure deserialization (`pickle.loads`, `yaml.load` without `SafeLoader`)
- Disabled TLS verification (`verify=False`, `rejectUnauthorized: false`)
- SQL built by string concatenation
- Weak hashing (md5/sha1) in a security context
- Secrets written to logs
- Path traversal from user input
- Sensitive files (`.env`, `.pem`, `.sql` backups, `id_rsa`)

How they work:

- Each rule owns its severity, so a match inherits HIGH/MED/LOW from its rule.
- Every rule runs a set of validators that suppress false positives and set a confidence score from 0 to 1. Severity reflects how serious the finding is, and confidence reflects how likely it is to be real, so the two are tracked separately.
- The dependency check (`--online`) reads `requirements.txt` / `package.json` and queries OSV.dev for known CVEs. It sends only the package name and version, never the source code.

2. What false positives could your scanner produce?

The validator layer suppresses the common false positives before they reach the report, like:

- Placeholder values: `api_key = "your_api_key_here"` is dropped by the placeholder validator.
- Environment lookups: `password = os.environ["DB_PASS"]` is dropped, because that's correct practice, not a hardcoded secret.

- Local and reserved URLs: `http://localhost`, `http://example.com`, and XML namespaces are allowlisted instead of flagged.
- Non-security hashing: md5/sha1 only fire when a security keyword (password, token, key) is nearby, so a cache key or ETag is left alone.
- Acknowledged findings: a line ending in `# nosec` is skipped.
- Comments and docstrings: `eval`, `exec`, `subprocess(..., shell=True)` and the other "code does X" patterns are dropped when they only appear in a `#` comment or a triple-quoted string, since that text isn't executable code.

What gets through after that is low-confidence by construction. The confidence score ranks it to the bottom, and `--min-confidence 0.6` removes it. One case the scanner reports on purpose rather than suppressing: an `http://` link inside a comment. It's down-weighted, but still listed, because a commented-out insecure endpoint is often worth a look.

3. What false negatives could it miss?

The scanner matches text line by line, so by design it does not catch:

- Secrets assembled at runtime or encoded, like `key = "ab" + "cd"` or a base64 blob.
- Patterns that span multiple lines.
- Injection where the tainted value arrives from another function. There's no data-flow analysis, so it doesn't trace input from source to sink.
- Content in binaries, files over 2 MB, or past the per-line length cap. The cap is deliberate, to stop a crafted line from stalling a regex.
- Dependency CVEs without `--online`, outside PyPI/npm, or in transitive dependencies.

These are the boundaries of a regex scanner. Question 4 covers what closes them.

4. How would you improve the scanner with more time?

Given the assignment scope, I focused on breadth of rules and a low false-positive rate rather than deep program analysis. A few things I deliberately left out, but would add with more time and a larger remit:

- Use a real parser (an AST, via tree-sitter) instead of regex for the data-flow rules (SQLi, command injection, path traversal). That would let me trace tainted input from where it enters to where it's used, even across functions, which regex can't do. That would close most of the false negatives in Q3.
- Calibrate the confidence weights against a labeled dataset and measure precision and recall, instead of setting them by hand.
- Extend the dependency check to more ecosystems and transitive dependencies, and cache and batch the OSV requests.
- Add a baseline mode so CI reports only findings that are new since the last run.
- Add exploit-aware prioritization: cross-reference CVEs against CISA's KEV list and EPSS scores to push actively-exploited issues to the top.
- Auto-tune the parallel threshold. File scanning already runs across processes, but the point where that beats a sequential pass depends on the machine (process startup is far cheaper on Linux than on Windows), so the fixed file-count cutoff could be measured at startup instead.

5. What security or privacy concerns exist when scanning source code?

Source code is sensitive and the scanner reads all of it, so:

- The report is sensitive output. It maps the weak points, and the evidence field can quote a real secret, so it shouldn't be committed. The `.gitignore` excludes the generated reports for that reason.
- The dependency check is the only feature that uses the network. It sends package names and versions to OSV, never code, and it's off by default.
- The HTML report embeds scanned content, so every value is HTML-escaped to stop a malicious file from injecting script into the page when it's opened.
- Scanning untrusted code means reading attacker-controlled input, which is why the line-length cap (ReDoS) and symlink containment (no following links outside the target folder) are in place.

6. How would you prioritize findings if this were used in a real company?

Severity on its own isn't enough to act on, so I'd stack a few signals on top of it:

- Severity times confidence first. A HIGH the scanner is sure about outranks a HIGH it's guessing at. The confidence score exists so nobody gets paged over noise.
- Whether it's actually being exploited. For dependency CVEs I'd check CISA's KEV list (is the bug known to be exploited in the wild), pull the CVSS score from NVD, and look at the EPSS probability for how likely it is to get hit. A CVE on the KEV list is top priority; a high CVSS with near-zero EPSS can wait.
- Where the code runs. A hardcoded secret in a production config is an incident. The same string in a throwaway dev script or a test fixture is a note. Dev vs staging vs prod changes the priority significantly.
- Whether it's internet-facing. A broad CORS policy or a debug flag on a public service matters far more than the same thing on an internal tool behind a VPN. The bigger the exposure, the higher the priority.
- Blast radius. Code that touches auth, payments, or user data outranks a static marketing page, even for the identical rule.

In practice I'd wire it into CI and fail the build only on high-severity, high-confidence findings, with everything else landing as a warning or a backlog item rather than a blocker. Suppression (the scanner honors a `# nosec` comment) lets a team accept a risk and move on. The goal is to surface the small set of findings that need action now, not a long list of low-severity items that get ignored.